Лабораторна робота № 6

Мета роботи:

Сформувати практичні навички реалізації логування в ROS2, навчитися здійснювати моніторинг стану системи, виявляти помилки та реалізовувати механізм повернення до стабільної версії (rollback).

Репозиторій з роботою: https://github.com/zakappaa/ROS2_Ginzburg

Хід роботи

ROS2-середовище активовано командами source, змінна ROS_DISTRO має значення humble. Запуск sensor_node у поточному вигляді й команди ros2 node list та ros2 topic list підтвердили наявність ноди /sensor_node та її топіків. Топіка /heartbeat на цьому етапі ще немає.

Реалізація логування

Логування додано в sensor_node (пакет sensor_pkg). Нода реєструє події трьох типів: запуск, отримання даних та помилки.

Для імітації нештатних ситуацій додано параметр fault_probability, який змінюється під час роботи командою ros2 param set. Повідомлення INFO, WARN та ERROR обмежено однією появою на секунду, щоб не перевантажувати консоль при частоті публікації 10 Гц.

Повідомлення рівнів INFO, WARN та ERROR, отримані при роботі ноди (WARN викликано командою ros2 param set /sensor_node threshold 3.0, ERROR - командою ros2 param set /sensor_node fault_probability 1.0), наведено на рис. 1.

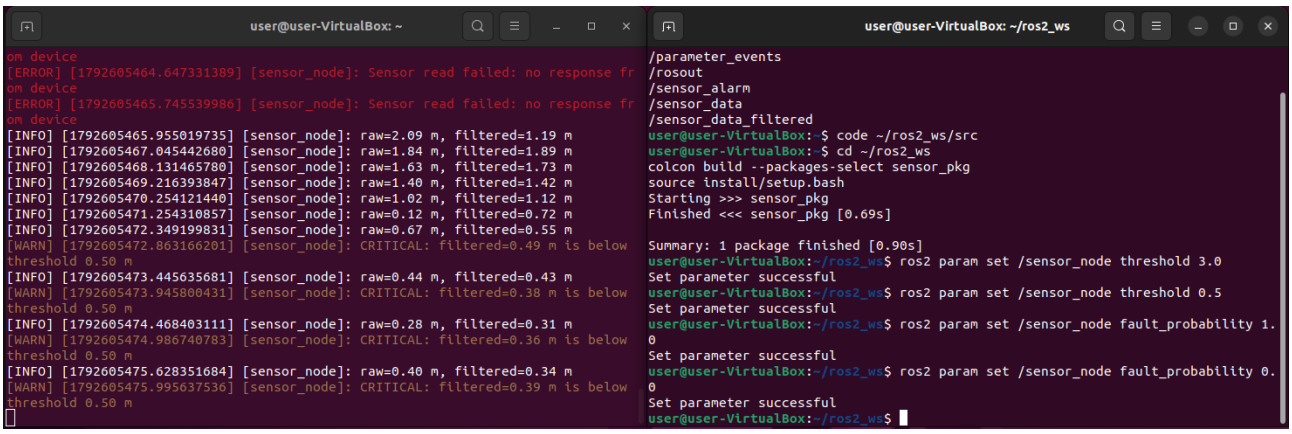


Рисунок 1 - Приклади логів рівнів INFO, WARN та ERROR

Рівнем виводу можна керувати без зміни коду: з ключем --ros-args --log-level debug додатково з'являються рядки DEBUG, а з --log-level warn - виводяться лише WARN та ERROR. Усі повідомлення автоматично зберігаються у файли.

Реалізація heartbeat

У sensor_node додано публікатор у топік /heartbeat і окремий таймер з періодом 1 с. Кожне повідомлення містить назву ноди та лічильник seq, що зростає на 1, тому за пропуском номерів видно втрату повідомлень.

Команди ros2 topic list -t, ros2 topic hz /heartbeat та ros2 topic echo /heartbeat підтвердили наявність топіка /heartbeat [std_msgs/msg/String], частоту +- <average rate з рис. 4> Гц і зростання лічильника (рис. 2).

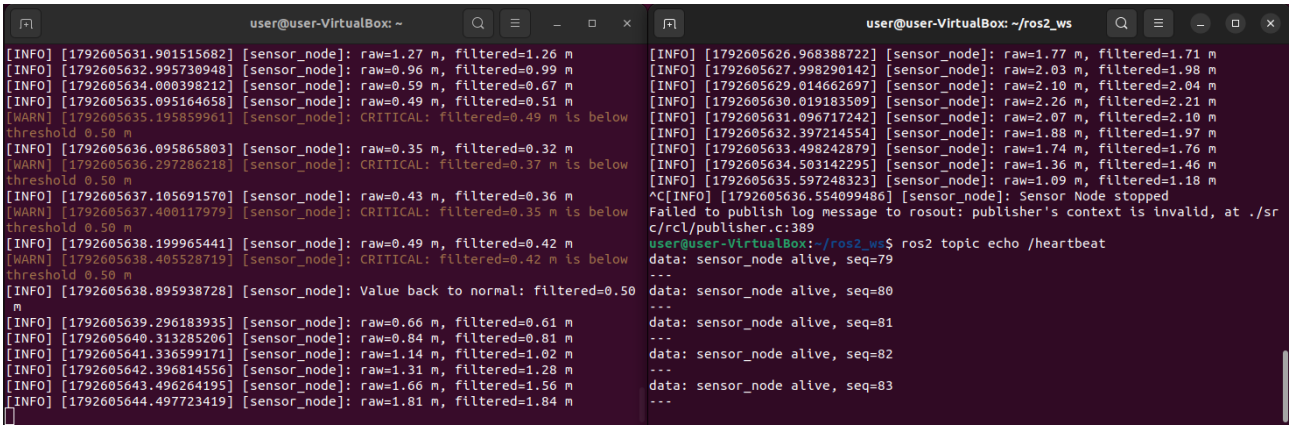


Рисунок 2 Heartbeat: тип топіка, частота та повідомлення

Моніторинг системи

Створення пакету monitor_pkg (рис. 3).

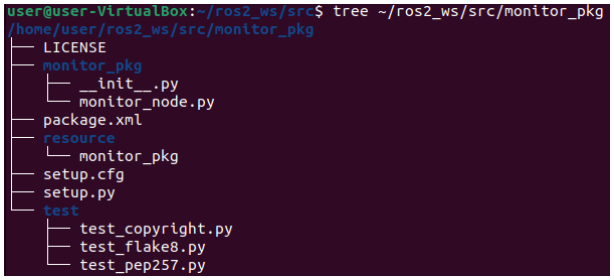


Рисунок 3 - Створення пакету monitor_pkg

Нода monitor_node підписується на /heartbeat і /sensor_data та раз на секунду оцінює стан системи. Результат виводиться в лог і публікується в топік /system_status.

Розрізнення випадків «нода жива, але даних немає» та «нода не відповідає» стало можливим саме завдяки heartbeat: за самими даними їх відрізнити неможливо.

Роботу монітора перевірено таких станах:

1. **Система стабільна.** Монітор виводить System stable: heartbeat ok, data rate 10.0 Hz, статус /system_status - OK (рис. 4).

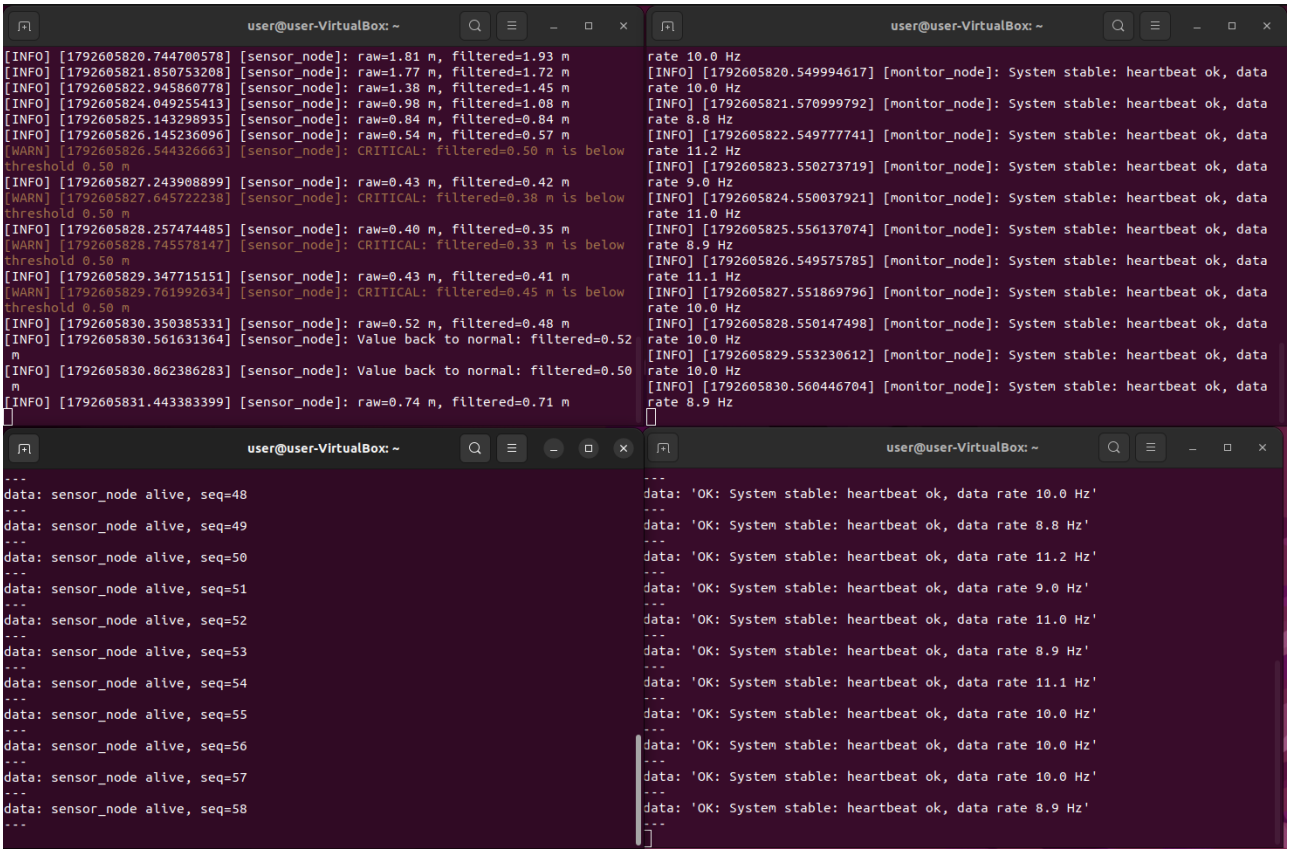


Рисунок 4 - Моніторинг стабільної системи

2. **Нестабільні дані та відсутність даних.** Командою ros2 param set /sensor_node fault_probability 0.5 спричинено збої читання: монітор вивів WARN Data unstable. При значенні 1.0 дані зникли, а heartbeat продовжував надходити, і монітор вивів ERROR No data on /sensor_data ... but heartbeat is alive (рис. 5).

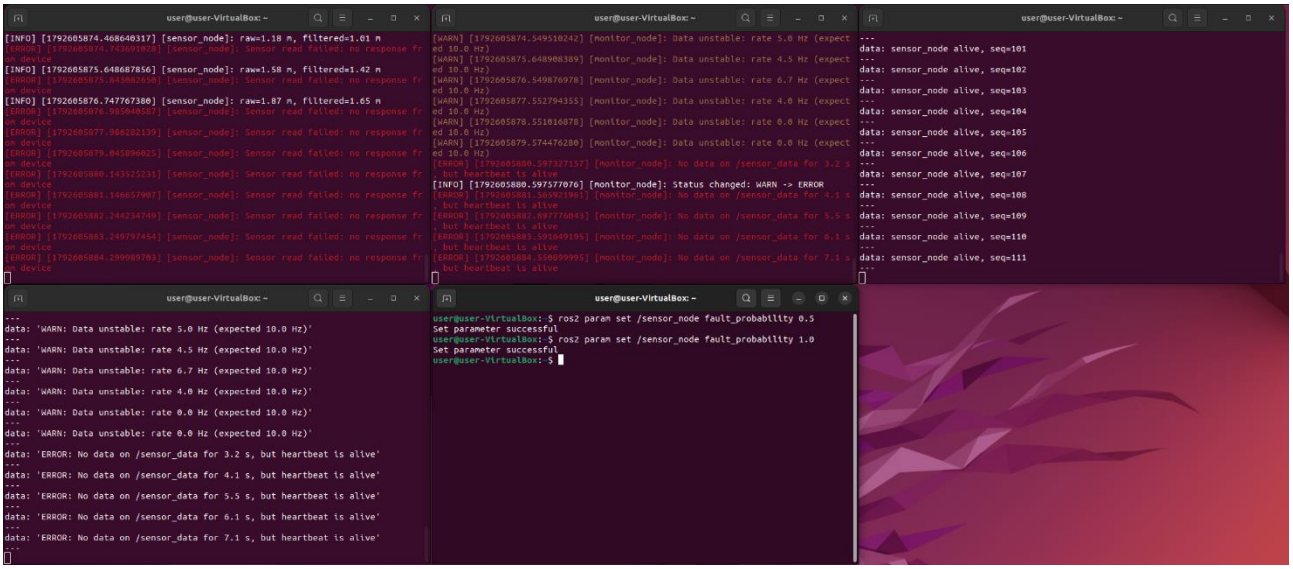


Рисунок 5 - Реакція монітора на нестабільні дані та їх відсутність

3. **Аварія ноди.** Після зупинки sensor_node (Ctrl + C) heartbeat припинився, і приблизно через 3 с монітор вивів ERROR Heartbeat lost (рис. 6).

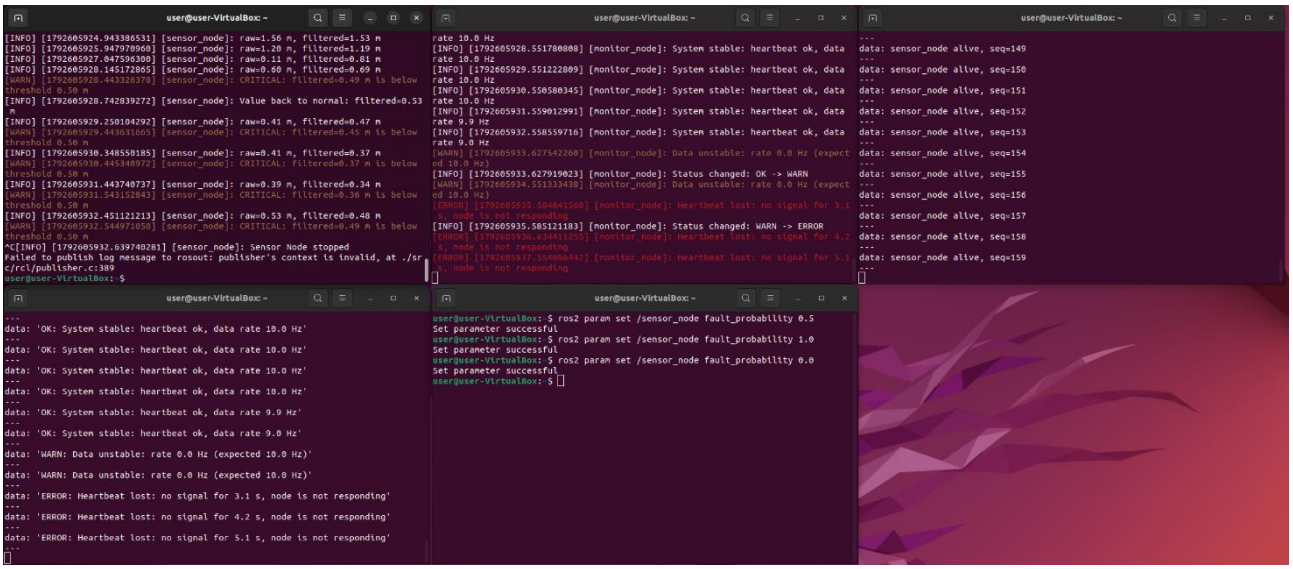


Рисунок 6 - Втрата heartbeat: нода не відповідає

Версіонування в Git

У каталозі ~/ros2_ws/src (де розташовано всі пакети) створено репозиторій Git. Службові файли Python виключено через .gitignore. Усі файли додано командою git add ., стабільну версію зафіксовано комітом (рис. 7).

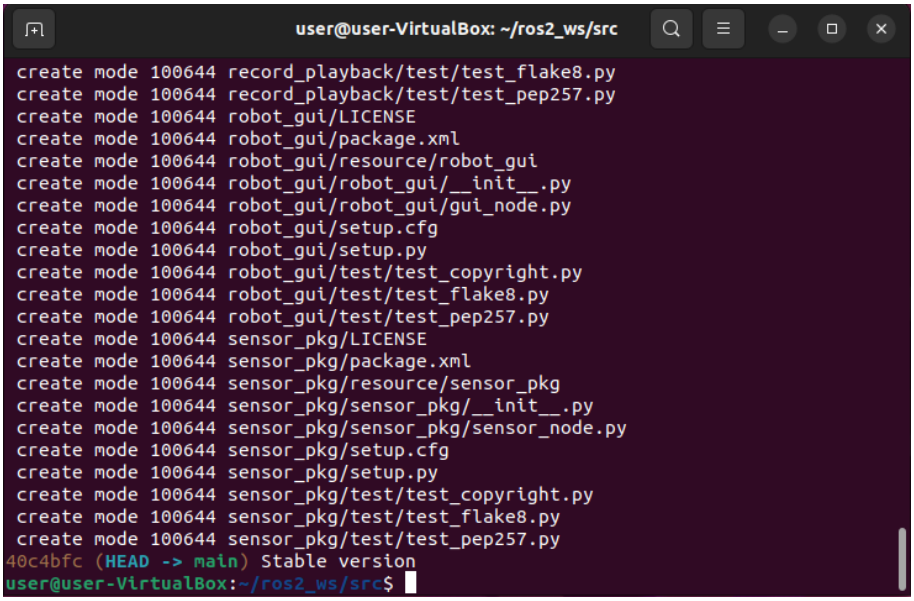


Рисунок 7 - Ініціалізація репозиторію та коміт «Stable version»

Rollback

Внесення помилки. У sensor_node.py змінено назву топіка публікації сирих даних: /sensor_data => /sensor_dat (типова одруківка). Зміну показано командою git diff (рис. 8).

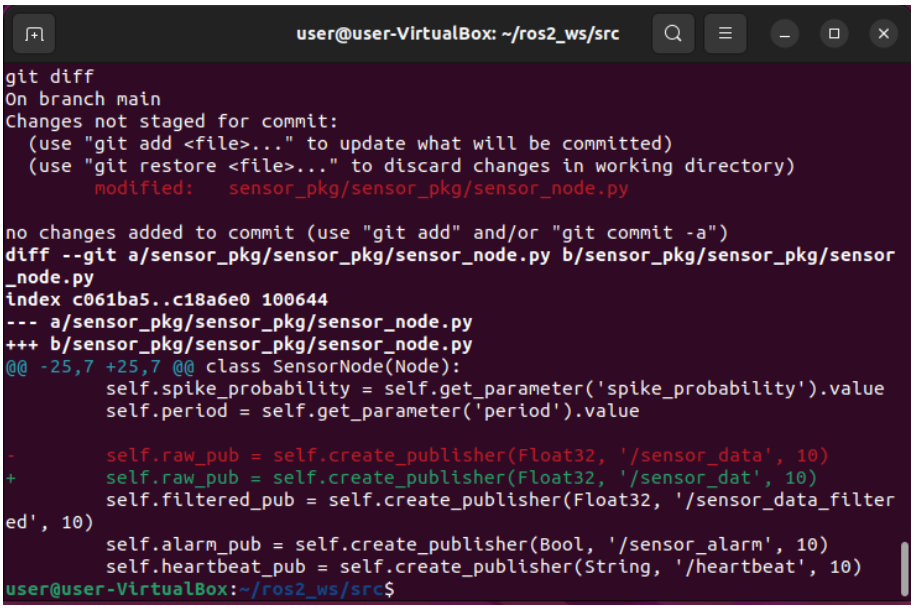


Рисунок 8 - Внесення помилки в код (git diff)

Некоректна робота системи. Після перезбірки нода sensor_node працює без помилок, але публікує дані не в той топік. Монітор виявив це: через 3 с з'явилося ERROR No data on /sensor_data ..., but heartbeat is alive, ros2 topic

list показує топік /sensor_dat, a ros2 topic hz /sensor_data нічого не виводить (рис. 9). Без моніторингу така помилка лишилась би непоміченою, бо нода не завершується аварійно й навіть пише у лог, що працює.

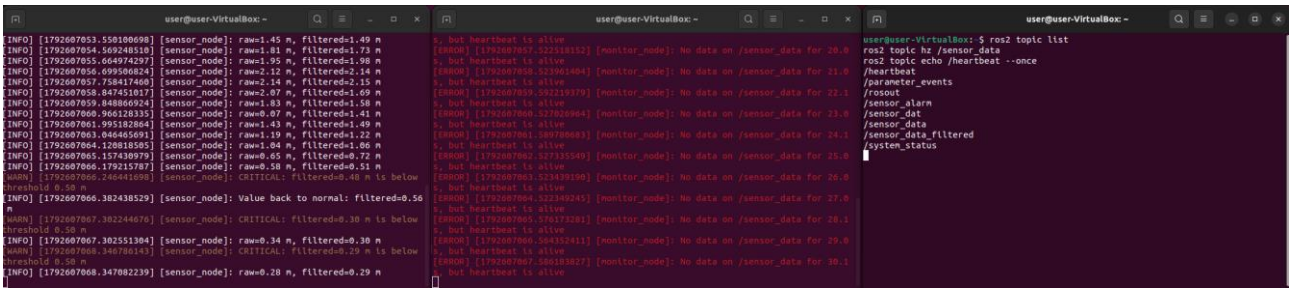


Рисунок 9 - Некоректна робота системи: дані не надходять у /sensor_data

Помилкову версію зафіксовано комітом, щоб HEAD~1 вказував на стабільну: git commit -am "Bug: wrong topic name"

Виконання rollback. Виконано команду git checkout HEAD~1. Git перемістив робочий каталог на коміт «Stable version» (стан detached HEAD), у файлі знову правильна назва топіка /sensor_data (рис. 10).

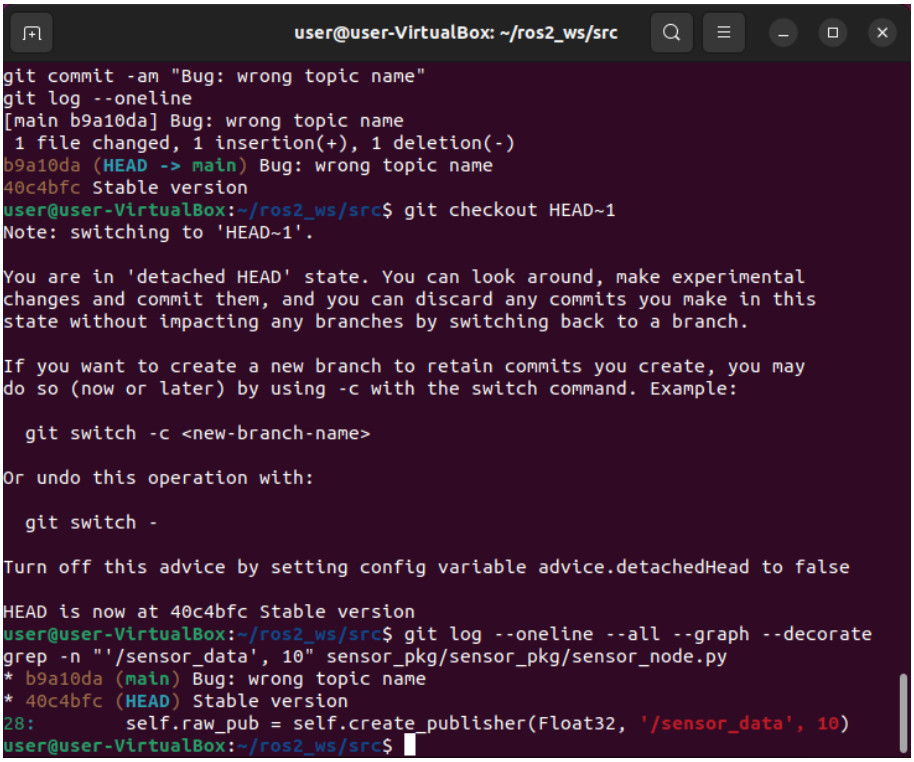


Рисунок 10 - Rollback: git checkout HEAD~1

Перевірка відновлення. Після colcon build та повторного запуску монітор вивів System stable: heartbeat ok, data rate 10.0 Hz, а ros2 topic hz /sensor_data показав +- <значення> Гц (рис. 11). Роботу системи відновлено.

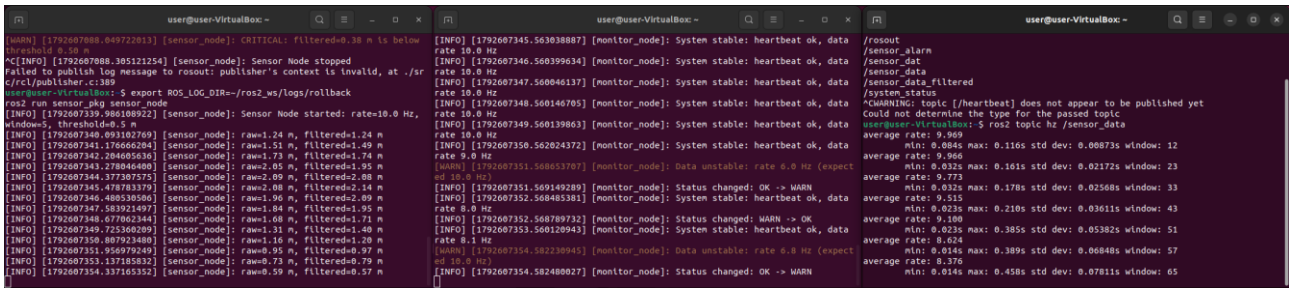


Рисунок 11 - Система працює коректно після rollback

Закріплення відкату. Щоб не залишатись у стані detached HEAD, виконано повернення на гілку main та git revert, що створило коміт, який скасовує помилку, і не змінює історію (рис. 12).

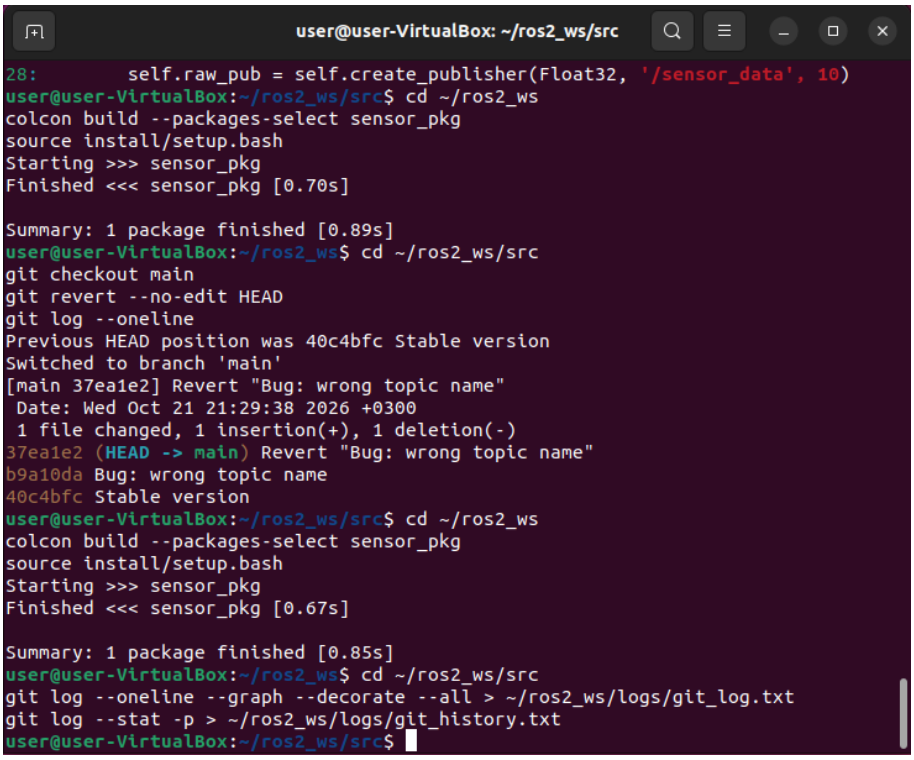


Рисунок 12 - Закріплення відкату: git revert та історія комітів

Висновок

У ході лабораторної роботи реалізовано логування в sensor_node з використанням рівнів INFO, WARN, ERROR (та DEBUG для діагностики) для запуску ноди, отримання даних і помилок, логи зберігаються у файли, а їх обсягом можна керувати під час запуску. Додано механізм heartbeat: нода щосекунди публікує у топік /heartbeat повідомлення типу std_msgs/msg/String з лічильником. Створено ноду-монітор, що за heartbeat і потоком даних визначає, чи працює нода, чи приходять дані та чи стабільна система, і розрізняє збої читання сенсора, відсутність даних за живої ноди та аварію ноди.

Код версіоновано в Git: стабільну версію зафіксовано комітом «Stable version». Навмисну помилку в коді (одруківка) виявив монітор, після чого система була відновлена командою git checkout HEAD~1, а відкат закріплено командою git revert.